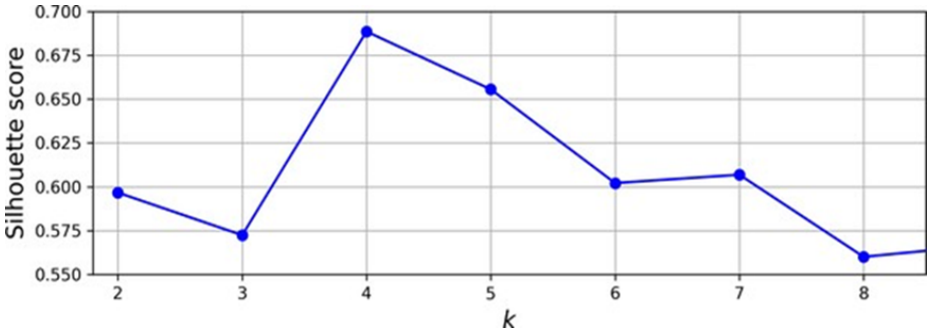
Đánh giá chất lượng bằng điểm Silhouette

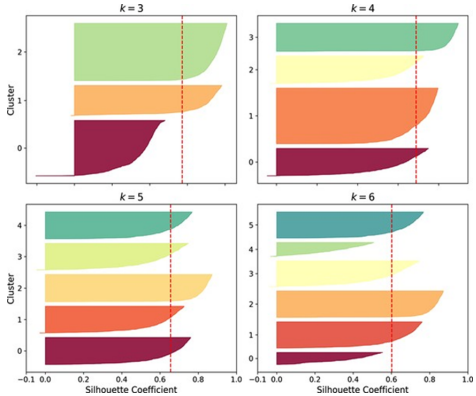
- Phương pháp "khuỷu tay" quá thô sơ. Đánh giá **điểm Silhouette (Silhouette Score)** chính xác hơn.
- Điểm Silhouette của mỗi điểm dữ liệu: $s = \frac{b-a}{\max(a,b)}$
  - a: Khoảng cách trung bình đến các điểm trong cùng cụm.
  - b: Khoảng cách trung bình đến các điểm trong cụm gần nhất tiếp theo.
- Giá trị dao động từ -1 đến $+1$.
- Gần $+1$: Điểm nằm sâu trong cụm.
- Gần 0: Điểm nằm ở rìa cụm.
- Gần -1: Điểm có thể bị gán sai cụm.

Biểu đồ điểm Silhouette theo k



Hình 9-9. Chọn số lượng cụm k tối ưu (đỉnh cao nhất, ví dụ $k = 4$ hoặc $k = 5$)

Phân tích biểu đồ dao (Silhouette plots)

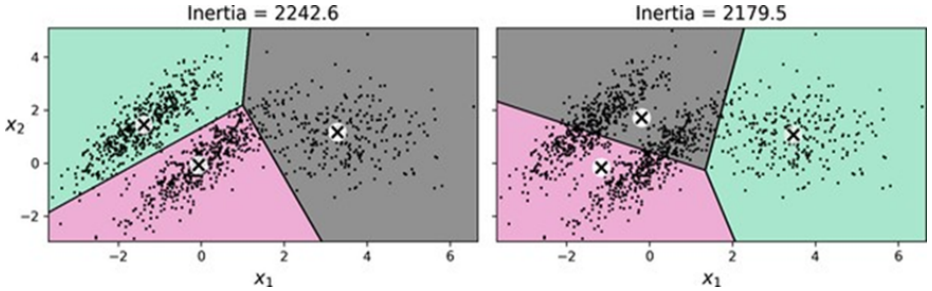


Hình 9-10. Hình dao cho mỗi cụm. Dao càng rộng, cụm càng tốt. Ranh giới đỏ là điểm Silhouette trung bình.

Giới hạn của K-Means

- Cần chạy thuật toán nhiều lần để tránh tối ưu cục bộ.
- Phải chỉ định trước số cụm k.
- **Nhược điểm lớn nhất:** K-Means hoạt động rất kém khi các cụm có kích thước khác nhau nhiều, mật độ khác nhau, hoặc hình dạng không phải hình cầu (ví dụ: hình elip).
- Lý do: K-Means chỉ quan tâm đến **khoảng cách** tới tâm cụm.

K-Means gặp khó khăn với các cụm hình elip



Hình 9-11. K-Means phân cụm sai lầm khi cắt ngang khối hình elip tự nhiên

Ứng dụng: Phân đoạn hình ảnh (Image Segmentation)

- Phân đoạn hình ảnh: Chia hình ảnh thành nhiều phân đoạn.
- Ứng dụng phổ biến: **Phân đoạn màu sắc**. Nhóm các pixel có màu giống nhau vào cùng một cụm K-Means.
- Thay thế giá trị màu của mọi pixel bằng màu của tâm cụm K-Means, ta có thể giảm số lượng màu xuống k màu, tạo hiệu ứng ảnh như tranh vẽ (Posterization).

Phân đoạn màu sắc hình ảnh bọ rùa

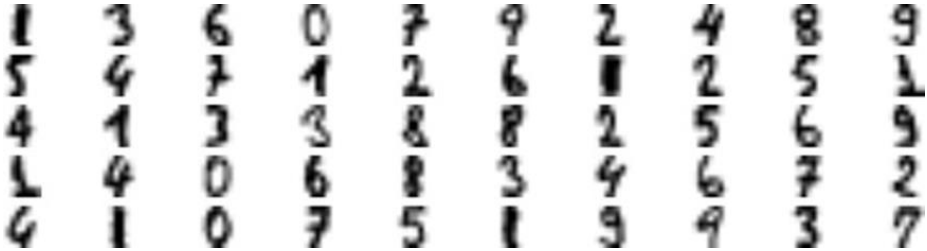


Hình 9-12. Phân đoạn hình ảnh sử dụng K-means với các số lượng cụm màu khác nhau ($k = 10, 8, 6, 4, 2$)

Ứng dụng: Tiền xử lý để học bán giám sát

- Khi bạn có rất nhiều dữ liệu nhưng chỉ một phần rất nhỏ có nhãn (vd: chỉ gán nhãn được 50 tấm ảnh MNIST).
- Thay vì gán nhãn 50 ảnh ngẫu nhiên, ta dùng K-Means chia tập dữ liệu thành 50 cụm.
- Sau đó, tìm hình ảnh **gần tâm cụm nhất** (hình ảnh đại diện) và chỉ gán nhãn cho 50 hình ảnh này.
- Cuối cùng, có thể **truyền lan nhãn** (Label Propagation) sang tất cả các ảnh khác trong cùng cụm.

Lấy 50 hình ảnh đại diện bằng K-Means



Hình 9-13. 50 hình ảnh chữ số đại diện (một hình ảnh cho mỗi cụm). Việc gán nhãn chúng mang lại ý nghĩa cao hơn chọn ngẫu nhiên.

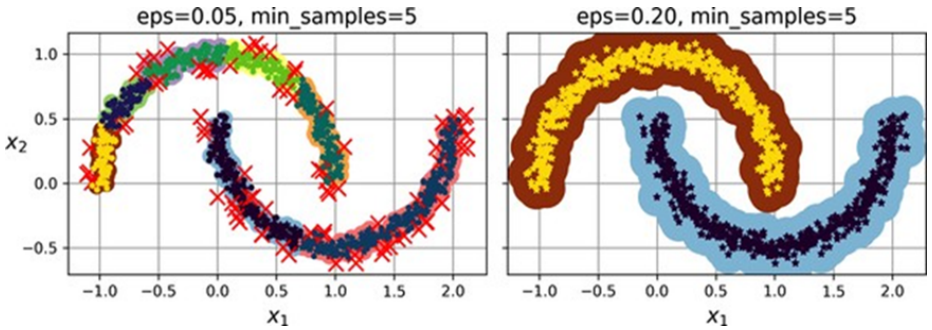
Thuật toán phân cụm DBSCAN

- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise):** Dựa trên ý tưởng các cụm là các khu vực có **mật độ cao** phân tách bởi khu vực có **mật độ thấp**.
- Không giống K-Means, DBSCAN xác định được các cụm có **hình dạng tùy ý** và có thể phát hiện điểm ngoại lai (nhiễu).

Cơ chế hoạt động của DBSCAN

- Nhìn vào bán kính khoảng cách ε (**eps**) xung quanh mỗi điểm dữ liệu. Khu vực này gọi là ε-*neighborhood*.
- Nếu khu vực này có chứa ít nhất **min_samples** điểm, điểm đó được coi là một **điểm lõi (core instance)**.
- Bất kỳ điểm nào nằm trong vùng lân cận của một điểm lõi cũng thuộc cùng cụm. (Các điểm lõi gần nhau kết hợp lại thành cụm lớn).
- Nếu điểm không phải là lõi và không nằm trong vùng của điểm lõi nào $\rightarrow$ Nó bị coi là Dị thường (Anomaly / Noise).

DBSCAN với 2 bán kính vùng lân cận khác nhau



Hình 9-14. DBSCAN bắt được hình dạng "mặt trăng" cực tốt. eps nhỏ (trái) tạo nhiều cụm đứt gãy. eps lớn hơn (phải) tạo ra 2 cụm hoàn hảo.

Ưu điểm và nhược điểm của DBSCAN

- **Ưu điểm:**
  - Không cần khai báo số cụm k trước.
  - Kháng nhiễu cực tốt (tự phát hiện và loại bỏ điểm ngoại lai).
  - Tìm được cụm có hình dạng hình học phức tạp (chữ U, hình vành khăn, mặt trăng...).
- **Nhược điểm:**
  - DBSCAN **không có** phương thức `predict()`, nó không thể tự gán cụm cho một trường hợp mới sinh ra.
  - Kém hiệu quả nếu các cụm trong tập dữ liệu có **mật độ khác biệt nhau quá lớn**.

Dự đoán với DBSCAN (Kết hợp KNN)

Vì DBSCAN không có `predict()`, ta có thể dùng thuật toán phân loại KNN để huấn luyện trên các lõi DBSCAN vừa tìm được.

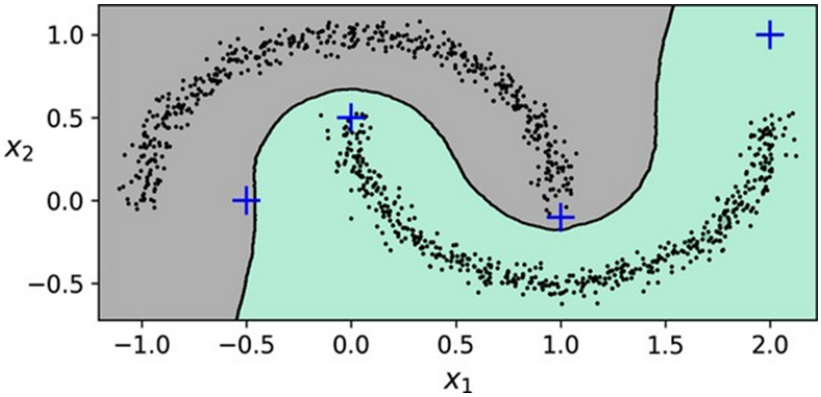
```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=50)
# dbscan.components_ chua cac diem loi
# dbscan.labels_ chua nhan cua cac diem do
knn.fit(dbscan.components_, dbscan.labels_[dbscan.core_sample_indices_])

import numpy as np
X_new = np.array([[-0.5, 0], [0, 0.5], [1, -0.1], [2, 1]])
print(knn.predict(X_new))
# Output: [1 0 1 0]

```

Đường biên quyết định của phân loại KNN trên tập DBSCAN



Hình 9-15. Dùng KNN tạo đường biên phân loại mới dựa trên các cụm đã tìm thấy bởi DBSCAN